Introductory Programming and Problem-Solving Skills

Week - 9

Modular Design

Basanta Singh

Some of the highlights

Navigation

Let's Recap

- Module Introduction and Computational Problem Solving
- Data And Expression
- Function
- Sequential And Conditional Statement
- Iteration Statement
- Data Structures
- File Handling and String Processing
- Exception Handling

Motivation

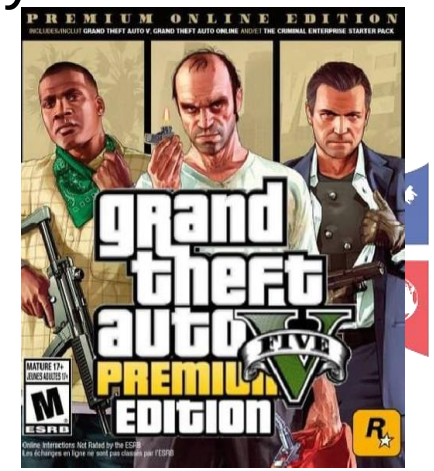
- **Previously, while programming we used to define various function in order to make our code optimise and use the benefit of concept, code – reusability. But at that time, we used to reuse our code within the same program but the question is can we re-use the entire python file in our next python file?**

Modular Design

But Before That, Let’s Understand its Importance.

Importance of Modular Programming

- **Imagine You're Developing a Game**. Games are typically made of different modules including **graphics, physics, AI and sound**. By developing each module separately, developers can ensure that each one works correctly and optimize them individual. This can lead to better performance and more enjoyable gaming experience for players.



What is a Module?

Introduction

Navigation

1. Module

- An important aspect of well-designed software is that program are designed as a collection of modules. **The term "module," broadly speaking, refers to the design and/or implementation of specific functionality to be incorporated into a program or program design.**

- While an individual function may be considered a module, modules generally consists of a collection of functions (or other entities).

- The Python turtle, math, random module are some of the example of software module

Module Specification

Module Specification

- **Every module needs to provide a specification of how it is used**. This is referred to as the module's interface. Module specification should be sufficiently clear and complete so that clients can effectively utilize it.

- [**Example**]: num_primes is a function that return how many prime numbers are in a given integer range, shown below:

```
def num_prime(start, end):
        '''Returns the number of primes between start and end'''
```

- As we know, the **function specification** is provided by the line immediately following the function header, called a **docstring** in Python. A docstring is a string literal denoted by triple quotes given as the first line of certain program elements.

More About Module Specification

- The docstring of a given program element can be displayed by use of the **__doc__** extension.

```
>>> print(num_primes.__doc__)
Returns the numbers of prime between start and end.
```

- This provides a **convenient way** for discovering how to use a given function without having to look at the function definition itself. Some software development tools also make use of docstrings.

- **For descriptive module specification**, (describing about the parameters and their conditions) you can use the concept of **multiline docstring** and document function in more detailed manner.

Note: A docstring can be both single line and multiline. Both the standards have their own use case.

Python Main Modules

Navigation

1. Module

1.2 Module Specification

2. Python Modules

- A **python modules** is a file containing python **definitions and statements**. Modules in python are named using all lowercase letters and optional underscore characters
- When a Python file is directly executed, it is considered the main module of a program. Main modules are given the special name **__main__.**
- Main modules provide the basis for a complete Python program. They may import (include) any number of other modules (and each of those modules may import other modules etc.)
- Python modules provide all the benefits of modular software design.

Note: Main modules are not meant to be imported into other modules

Types of Modules

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

We can consider by saying that there can be two types of modules in python.

I. **Python Standard Modules:** The Python Standard Library contains a set of predefined Standard **(built-in) modules**. We have seen some of these modules already, such as the math, date and time, random Standard Library Modules.

I. **User-Defined Modules:** User-defined modules are simply python scripts that consist of classes, functions and other objects that will be developed **as per user's requirement** and can be reused in other python scripts.

Let's Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

Let's Understanding

- Here, we are using factorial function from math python standard module.

```
#module main (main.py)
import math
a = int(input("Enter a Number to find a fact:"))
print(math.factorial(a))
```

```
Enter The Number You Want To Find Factorial: 5
120
```

Let's Understand Through Example

Navigation
1. Module
1.2 Module Specification
2. Python Modules
2.1 Types of Modules
Let's Understanding

- Create a Python module by entering the following in a file name simple.py. Then execute the instructions in the Python shell as shown

```
#module simple (simple.py)
print('module simple loaded')

def func1():
    print('func1 called')

def func2():
    print('func2 called')
```

```
>>> import simple
module simple loaded
>>> simple.func1()
func1 called
>>> simple.func2()
func2 called
>>> |
```

What if two or more python modules contain identifiers with the same name and are imported in same program??

For That, Let’s Understand Modules and Namespaces

What is Namespace?

Namespaces

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

- [**Scenario**]: Suppose we have three same name of students in Level 4, Level 5 and Level 6 which is "Samir". When they are inside a class then there is not a problem to identify. But **what if we just call the name "Samir" in spring carnival?** Basically when we call their name then the **ambiguity starts** and name space avoids such problems.

- **Namespace** [**Definition**]: A namespace is a **container** that provides a named context for a set of identifiers. Namespace avoids enable programs to avoid potential **name clashes** by associating each identifier with namespace form which it originates.

Let's Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

Example


```
# module1

def double(lst):

    """Returns a new list with each
       number doubled, for example,
       [1, 2, 3] returned as [2, 4, 6]
    """
```

```
# module2

def double(lst):

    """Returns a new list with each
       number duplicated, for example,
       [1, 2, 3] returned as
       [(1, 1), (2, 2), (3, 3)]
    """
```

```
import module1
import module2

# main
  .
  num_list = [3, 8, 14]
  result = double(num_list)
  .
```

ambiguous reference for identifier `double`

To Solve Such Problem..

Modules and Namespaces

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

Example

- In Python, each module has its own namespace.
- This includes the names of all items in the **module**, including **functions and global variables** – variables defined within the module and outside the scope of its functions.
- The two instances of identifier double, each defined in their own module, are distinguish by being fully qualified with the name of the module in which each defined:

module1.double and module2.double

Let’s Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

Example


```
import module1
import module2

# main

   ans1 = module1.double(...)

   ans2 = module2.double(...)
```

references function double from module1’s namespace

references function double from module2’s namespace

Let’s Try

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

TryOut

Enter each of the following functions in their own module named mod1.py and mod2.py, then enter and execute the following in the shell.

```
# mod1
def average(lst):
    print('average of mod1 called')

# mod2
def average(lst):
    print('average of mod2 called')
```

```
>>> import mod1, mod2
>>> mod1.average([10, 20, 30])
???
>>> mod2.average([10, 20, 30])
???
>>> average([10, 20, 30])
???
```

Now, Let’s Look at Different Ways That Python Modules Can Be Imported

Importing Module

Importing Modules

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

- The main module of any program is the first ("top-level") module executed.
- When working interactively in Python shell, the Python interpreter functions as the main module, containing the global namespace.

Remember: The namespace resets every time the interpreter is restarted.

Note: Module __builtins__ is automatically imported in Python programs, providing all the built-in constants, functions, and classes.

Let’s Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

Example


```
#module simple (simple.py)
print('module simple loaded')

def func1():
    print('func1 called')

def func2():
    print('func2 called')
```

```
>>> import simple
module simple loaded
>>> simple.func1()
func1 called
>>> simple.func2()
func2 called
>>> |
```

Importing Module Functionality

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

4.1 Importing Functionality

- When using the import <***modulename***> form of import, the namespace of the imported module becomes available to, but not part of, the importing module.
- Identifiers of the imported module, therefore, must be fully qualified (**prefixed with the modules name**) when accessed. Using this form of import prevents any possibility of a name clash.
- Thus, as we have seen, if two modules, module1 and module2, both have the same identifier, identifier1, then module1.identifier1 denotes the entity of the first module and module2.identifier1 denotes the second entity of the second module.

from-import

The “from-import” Form of Import

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

4.2 from import module

- Python also provides an alternate import statement of the form which is:

from <modulename> import something

Something can be list of identifiers, a single renamed identifier, or and asterisk.

a. from ***modulename*** import func1, func2
b. from ***modulename*** import func1 as new_func1
c. from ***modulename*** import *

Let's Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

4.2 from import module Example


```
from math import factorial

a = int(input("Enter a number:"))
print(factorial(a))
print(pi)
```

```
Enter a number: 5
120
Traceback (most recent call last):
  File "E:\Herald Teaching Resources\IPPS - S23\pythonProject\venv\week11_main.py", line 5, in <module>
    print(pi)
NameError: name 'pi' is not defined
```

When using from-import, **the imported module's namespace becomes part of the importing module's namespace**. Thus, imported identifiers are referenced without being fully qualified

>>> factorial(a)

Let's Understand Through Example

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

4.2 from import module Example


```
#mod1
def area():
  print("Area of Rectangle")
#mod2
def area():
  print("Area of Triangle")
```

```
from mod1 import area as area_rectangle
from mod2 import area as area_triangle

print(area_rectangle())
```

The from <***modulename***> import area as area_triangle form of import is use when identifiers in the imported module's namespace are known to be identical to identifiers of the module, which would otherwise cause naming clashes.

Module Loading and Execution

Navigation

1. Module

1.2 Module Specification

2. Python Modules

2.1 Types of Modules

3 Namespace

4. Importing Modules

4.2 from import module

4.3 Module Loading and Execution

- Each imported module needs to be located and loaded into memory. Python searches for modules in the following order:

a. The current directory
b. The directories listed in PYTHONPATH environment var
c. A Python installation specific directory (e.g., C\Python32\Lib)

Note: If still not found, an error (ImportError exception) is reported. However, for now, all of the modules of a program will be kept in the same directory.

Have Some Research!!

Navigation

Have Some Research

- Explore the Use case of *from* ***modulename*** *import**
- What does built-in dir() function do?
- List down the difference between all form of import.

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing